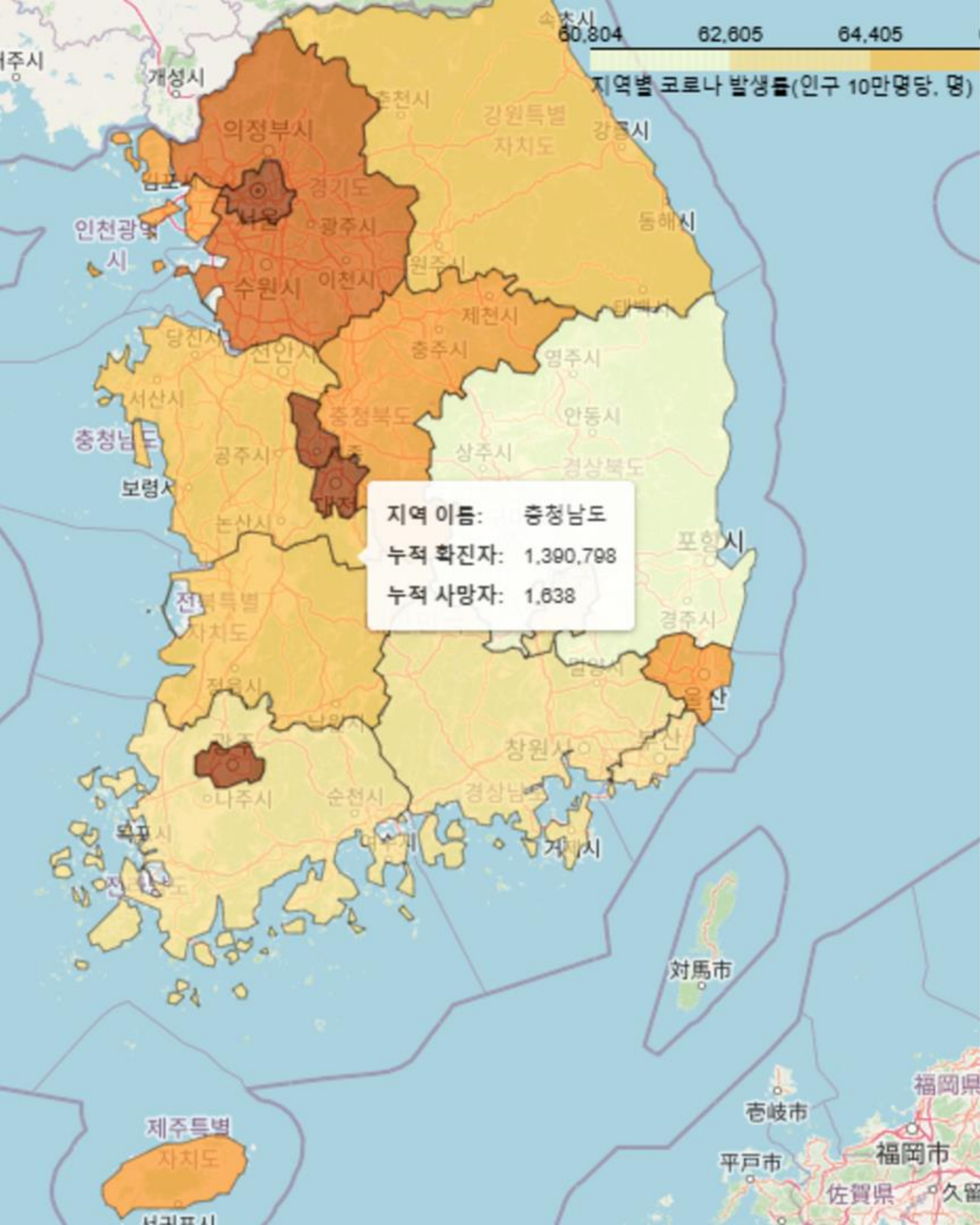




코로나 맵(Corona Map)

코로나 맵은 코로나19 국내 및 국외 데이터를 시각화하여 제공하는 맵 기반 웹 애플리케이션입니다. Flask를 활용하여 서버를 구축하고, 사용자 친화적인 UI를 제공합니다.

국내 COVID-19 데이터 시각화

데이터 분석

- 각 지역별 인구수 비례 코로나 발병률, 사망률, 완치율 비율 시각화
- 연령대별 코로나 발병률, 연령대별 사망률 중 확진자에 해당하는 사망률 분석

종합 통계 제공

- 각 지역별 전체 확진자 및 사망자 누적 수
- 월별 평균 확진자 수와 사망자 수 제공

보건소 정보 활용

- 국민안심병원 및 선별진료소 현황 데이터를 기반으로 관련 정보 제공

연령별 인구 데이터 연계

- 연도별 연령별 인구 데이터를 활용하여 지역 및 연령대별 감염률 비교

관련 뉴스 제공

- 각 지역과 연관된 최신 뉴스 기사 표시

국외 COVID-19 데이터 시각화

글로벌 현황 제공

- 전 세계 일간 데이터(어제, 오늘, 내일) , 누적 데이터 표기

날짜별 국가별 현황

- 특정 날짜에 따른 각국의 감염 데이터 및 감염 비율 분석

기간별 통계 제공

- 국가별 일간, 주간, 월간 감염 추세를 파이, 막대, 라인 그래프로 시각화

관련 뉴스 제공

- 각 국가와 연관된 최신 뉴스 기사 표시

구분	아이디	요청주소 (전부 GET)	설명
국내	main	/	전국 페이지에 관련된 모든 데이터들을 반환
	graph	/api/graph/<graph_id>	전국 페이지에 들어갈 그래프의 데이터들과 그래프를 반환
	area	/<area>	area 즉, 해당(특정) 지역 페이지에 관련된 모든 데이터들을 반환
	area_graph	/api/graph/<graph_id>/<area>	해당 지역 페이지에 들어갈 그래프의 데이터들과 그래프를 반환
	summary	/api/covid-summary	지역별 코로나 완치율에 관련된 데이터들과 그래프를 반환
국외	world_main	/worldwide	기본값(2년 6개월 전) 전세계 코로나 데이터, 국가명, 감염률 html에 담아 render_template
	world_marker	/worldwide/request/marker-data	각 국가의 위,경도를 담은 데이터 반환
	world_covid_data	/worldwide/covid-data/<date_type>	요청 날짜, 타입에 해당 하는 전세계 일간 감염데이터, 전날대비 데이터, 예측결과 반환
	world_daily	/worldwide/get-daily-data	요청 날짜에 해당하는 특정 국가의 일간, 주간, 월간 감염 데이터를 딕셔너리 형태로 반환
	world_data	/worldwide/data	요청 날짜의 전세계 코로나 데이터 반환
	world_db	/worldwide/insert-data	2020.01.04~2024.11.17 코로나 데이터 DB에 삽입
		/worldwide/insert-trans-data	한글 번역된 나라이름 db에 삽입
		/worldwide/insert-latlong-data	각 국가의 위,경도 값 db에 삽입

03 역할분담

국내

최하영

- 국내 페이지 와이어프레임 제작
- 국내 코로나 데이터 수집, 가공.
- 국내 코로나 관련 기사 크롤링
- 지도에 시각화 자료 삽입 및
각 지역페이지로 이동할 수 있는 기능 추가
- 각종 데이터 가공 후 시각화
- 보건소 API를 활용하여 의료기관 관련 정보 추가
- 국내 페이지 전체적인 UI 제작

권민수

- 국내 코로나관련 데이터 수집
- 국내 페이지 완치율 관련 기능 구현
- 완치율은 명확한 지표가 없어,
격리 해제를 기준으로 비율 구현

국외

이영찬

- 팀 관리 및 프로젝트 전반적인 설계
- 국외 페이지 자료 조사 및 UI 구현
- 데이터 DB 저장 및 국외 API 설계
- 국외 페이지 와이어프레임 제작
- Notion을 통한 팀 프로젝트 관리
- GitHub 관리, 초기 세팅
- README 작성, 발표 PPT 준비

주요 기능 담당:

- 어제, 오늘, 내일, 전세계 일간 데이터 및
누적현황 제공
- 내일의 코로나 감염 예측 모델 구현
- 각 국가별 감염 리스트 및
감염 비율 제공 및 지도기능
- 각 국가별 일간, 주간, 월간 감염 통계 시각화

허경

- 국외 나라별 코로나 관련 기사 크롤링

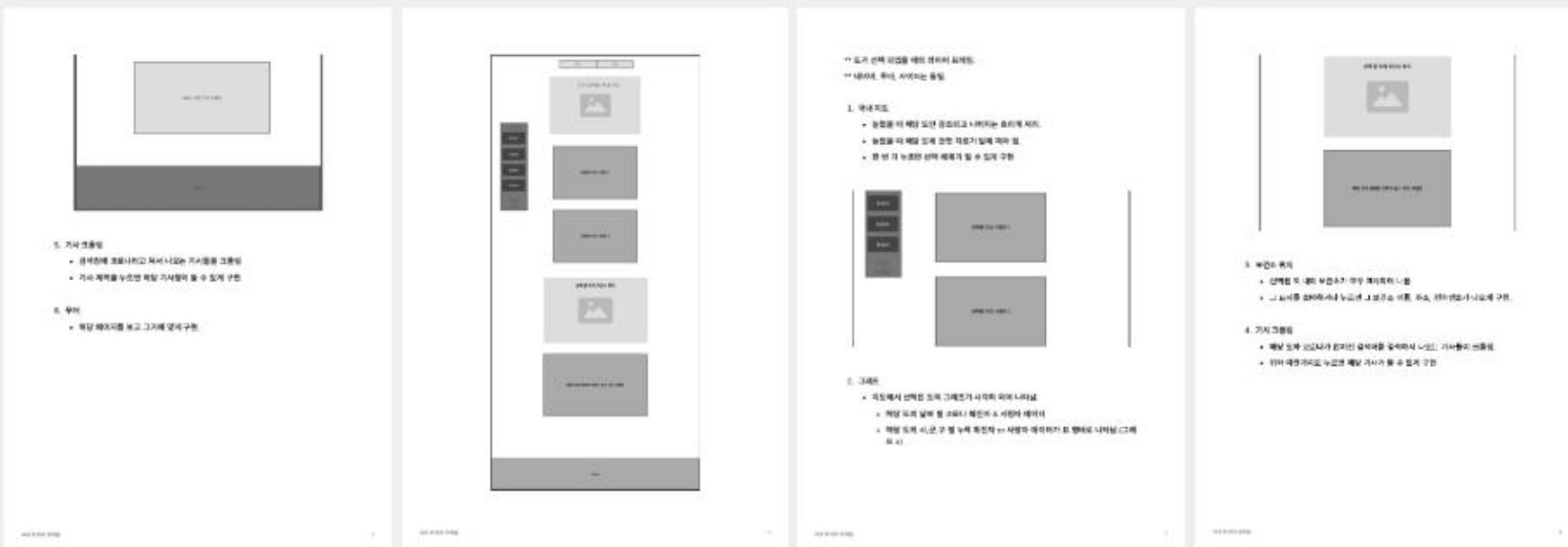
04 작업 기간 및 주요 내용

작업 기간: 2024년 12월 9일 ~ 2024년 12월 27일 (3주)

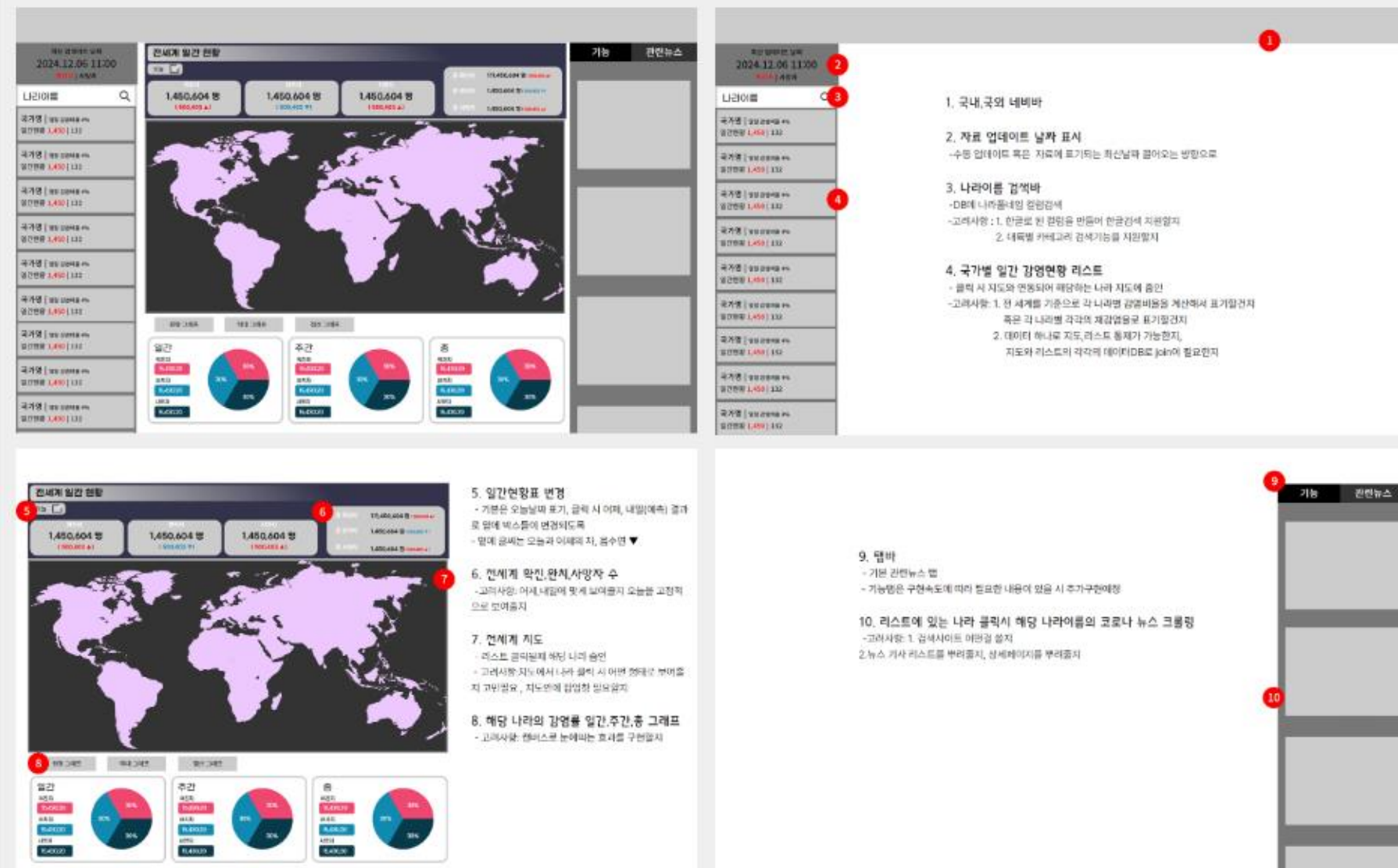
팀	1주차	2주차	3주차
이영찬	- 와이어프레임 제작 및 데이터 수집, 가공 - 국가 리스트를 데이터베이스에 저장하고 화면에 표시 - 국가 리스트와 지도 데이터 연동 작업	- 국가 리스트와 그래프 데이터 연동 작업 - 코로나 감염 예측 모델 생성 및 기존 코드와 통합	- UI 개선 및 관련 JavaScript 기능 구현 - 날짜별 데이터 조회 기능 추가를 위한 코드 최적화
최하영	- 와이어프레임 제작 및 데이터 수집, 가공 - 페이지 레이아웃 설계, 지도 삽입, 그래프 구조 설계 및 시각화 작업 - 국내 코로나 관련 기사 데이터 크롤링	- 지도 기능 추가 및 상세 기능 구현 - 데이터 기반 그래프 시각화 분류 작업	- 지역 클릭 시 상세 데이터를 보여주는 지도 기능 개발 및 콘텐츠 추가 - 보건소 API를 활용한 데이터 연동 작업
권민수	- 자료 조사 및 데이터 수집, 외부 api수집	- 데이터 가공 - Chart.js 활용 방안 구상	- 완치율 그래프 구현
허 경	- 국외 페이지 나라별 뉴스 크롤링	- 국외 페이지 나라별 뉴스 크롤링	- 국외 페이지 나라별 뉴스 크롤링

05 와이어 프레임

국내



국외



01

```
// '오늘', '어제', '내일', '내일(예측) 코로나 데이터 호출 및 로딩중 표시
document.addEventListener('DOMContentLoaded', () => {

// 기본값 설정
let selectedDateType = 'today';

const dateSelect = document.querySelector('#date-select');
const elementsToLoad = document.querySelectorAll('[data-loading-target]');

dateSelect.addEventListener('change', async (event) => {
  selectedDateType = event.target.value;

  // svg 삽입
  elementsToLoad.forEach(element => {
    element.innerHTML = getLoadingSVG();
  });
  // 데이터 로드

  await fetchCovidData(selectedDateType, getDateOnly());
});
// 초기 데이터 로드 (오늘 상태로)
fetchCovidData(selectedDateType);
});
```

- 페이지 로드 시 기본세팅값을 함수로 보내 서버에 기본 데이터 요청

- 클라이언트가 선택한 선택박스 값에 따라 데이터 호출

- 데이터 받아오기 전 화면에 보여줄 로딩svg를 삽입

- 클라이언트가 설정한 선택박스 기준과 현재 데이터를 조회한 날짜를 함께 보내 해당하는 데이터 서버에 요청

02

```
// 서버에서 COVID-19 전세계 일일 데이터를 가져오는 함수
async function fetchCovidData(dateType, date) {

  try {
    const formattedDate = (!date) ? '' : date;
    const response = await fetch(`/worldwide/covid-data/${dateType}?date=${formattedDate}`);

    if (!response.ok) {
      throw new Error(`Error fetching data: ${response.statusText}`);
    }
    const data = await response.json();
    // console.log(data)

    // 데이터를 화면에 반영하는 함수 호출
    updateCovidData(data);
  } catch (error) {
    console.error("Error fetching data:", error);
    console.log("fetchCovidData함수를 통해 데이터 호출 실패");
  }
}
```

```
function updateCovidData(data) {
  const fields = [
    { selector: '.new-cases', value: data.new_cas
    { selector: '.new-recoveries', value: data.ne
    { selector: '.new-deaths', value: data.new_de
    { selector: '.total-cases', value: data.total
    { selector: '.total-recoveries', value: data.
    { selector: '.total-deaths', value: data.tota
  ];
}
```

- 받아온 날짜가 없으면 undefined로 받고 공백으로 변경 후 전송

- 서버에 요청한 데이터를 json형태로 받음

- 받아온 데이터를 화면에 뿌려주는 함수로 전송

- 받아온 데이터를 객체의 배열 형태로 저장하여 활용

- 서버에서 URL파라미터와 쿼리스트링으로 받고 데이터를 조회하는 함수 호출

- 기본날짜 설정과, 받아온 날짜 기준으로 날짜 데이터를 잡고 타입에 따라 날짜를 한번더 갱신시켜줌

- 타입에 맞는 날짜 기준으로 DB조회 및 연산 후 데이터를 반환하며 예측타입은 예측모델 활용함수를 이용

03

```
# 전세계 데이터 계산 후 리턴
@worldwide_bp.route('/covid-data/<date_type>', methods=['GET'])
def get_covid_data(date_type):
    date = request.args.get('date')
    try:
        data = get_covid_data_for_date(date_type, date)
        return jsonify(data)
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

04

```
if not date:
    base_date = datetime.datetime.now().date() - timedelta(days=365 * 2 + 180)
else:
    base_date = datetime.datetime.strptime(date, '%Y-%m-%d').date()

# 날짜 타입에 따른 날짜 계산
date_mapping = {
    "today": base_date,
    "yesterday": base_date - timedelta(days=1),
    "tomorrow": base_date + timedelta(days=1),
    "prediction": base_date + timedelta(days=1), # 예측할 날짜
}
```

```
selected_date = date_mapping[date_type]
```

05

```
def _get_actual_data(date):
    today_data = get_total_data_for_date(date)
    yesterday_data = get_total_data_for_date(date - timedelta(days=1))

    return _calculate_changes(today_data, yesterday_data)

# 예측 데이터 반환 함수
def _get_predicted_data(date):
    today_data = predict_covid(date)
    yesterday_data = get_total_data_for_date(date - timedelta(days=1))

    return _calculate_changes(today_data, yesterday_data)

# 변화량 계산 함수
def _calculate_changes(today_data, yesterday_data):
    return {
        "new_cases": today_data["new_cases"],
        "new_cases_change": today_data["new_cases"] - yesterday_data["new_cases"],
        "new_recoveries": today_data["new_recoveries"],
        "new_recoveries_change": today_data["new_recoveries"] - yesterday_data["new_rec
        "new_deaths": today_data["new_deaths"],
        "new_deaths_change": today_data["new_deaths"] - yesterday_data["new_deaths"],
        "total_cases": today_data["cumulative_cases"],
```

```

1 # 예측 함수
2 def predict_covid(date_str):
3     try:
4         # 예측 날짜 처리
5         01 date_to_predict = pd.to_datetime(date_str)
6
7         # 날짜 정수화 (훈련 데이터 기준)
8         02 date_int = (date_to_predict - pd.to_datetime('2020-01-04')).days # 훈련 시작 날짜 기준
9
10        # 실제 데이터에서 전날 데이터 가져오기
11        03 covid_data_yesterday = get_total_data_for_date(date_to_predict - timedelta(days=1))
12        previous_day_data = np.array([
13            04 float(covid_data_yesterday[key]) for key in ['new_cases', 'new_deaths', 'new_recoveries', 'cumulative_cases', 'cumulative_recoveries']
14        ]).reshape(1, -1)
15
16        # 날짜 정보 추가
17        date_array = np.array([[date_int]]) # 날짜 정보 2D 배열로
18        # 입력 데이터 결합 (날짜 정보 + 전날 데이터)
19        05 input_data = np.hstack([date_array, previous_day_data]) # 차원이 일치해 결합 가능
20        # 입력 데이터 정규화
21        06 input_scaled = scaler_X.transform(input_data)
22
23        # 예측
24        predicted_scaled = model.predict(input_scaled)
25        predicted = scaler_y.inverse_transform(predicted_scaled) # 원래 스케일로 복원
26        predicted = np.maximum(predicted, 0) # 음수는 0으로 처리
27        07
28
29        # 예측된 값을 Decimal로 변환하여 반환
30        return {key: Decimal(str(predicted[0][i])).quantize(Decimal('1.')) for i, key in enumerate(
31            ['new_cases', 'new_deaths', 'new_recoveries', 'cumulative_cases', 'cumulative_recoveries',
32
33
34    except Exception as e:
35        print(f"오류 발생: {e}")

```

- 학습된 모델의 형식에 맞게끔 데이터 가공

1. 날짜를 pandas 날짜 타입으로 변환
2. 날짜를 정수화
3. 전날과 비교예측하기 때문에 전날데이터 db조회
4. 딕셔너리 데이터를 정제하고 numpy 배열화하여
모델에 입력으로 사용할 준비
5. 날짜와 입력데이터 결합 (1,6에서 1,7됨)
6. 0~1로 만드는 MinMaxScaler 데이터가 들어있는 스케일러 호출 후
데이터 정규화
7. 예측시작 및 스케일복원,음수 0처리 작업 후
기존 데이터 반환구조인 Decimal로 재가공 후 리턴

01

```
// -----검색 기능

// 폼 제출 시 검색 처리 (엔터,아이콘 클릭)
document.getElementById('search-form').addEventListener('submit', f
  event.preventDefault(); // 기본 폼 제출 방지
  searchCountries();
});
document.getElementById('search-input').addEventListener('input', f
  // 검색어가 비었으면 전체 리스트를 다시 표시
  if (this.value === '') {
    showAllCountries();
  }
});

const countryName = item.querySelector('p strong').textContent.toLowerCase(); // 첫 번째 p 안의
const countryEnglishName = item.querySelector('.country-english').textContent.toLowerCase(); //
if (countryName.includes(searchTerm) || countryEnglishName.includes(searchTerm)) {
  item.classList.remove('hidden'); // 일치하는 항목 표시
} else {
  item.classList.add('hidden'); // 일치하지 않는 항목 숨김
}
```

02

```
// 데이터를 병렬로 fetch
Promise.all([
  fetch('/worldwide/request/marker-data').then(res => res.json()),
  fetch('/worldwide/static/data/world_countries.json').then(res => res.json()),
])
.then(([markerData, geojsonData]) => {
  // 맵 초기화
  const map = L.map('map-container').setView([20, 0], 2);
  L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
    maxZoom: 19,
    attribution: '© OpenStreetMap contributors',
  }).addTo(map);
```

03

```
// 전역 변수
let chartType = 'pie'; // 초기 그래프 타입
let pieCharts = []; // 생성된 그래프를 저장하는 배열

// 그래프 생성 함수
function createCharts(data) {
  // 그래프 초기화
  pieCharts.forEach(chart => chart.destroy());
  pieCharts = [];

  const graphViews = document.querySelectorAll('.graph-view');
  const periods = ['daily', 'weekly', 'monthly'];

  periods.forEach((period, index) => {
    const graphView = graphViews[index];
    const { new_cases, new_recoveries, new_deaths } = data[period];

    // 데이터 변환
    const sanitizedData = [new_cases, new_recoveries, new_deaths].map(value => {
      value = Number(value);
      return value === 0 ? 0.1 : value; // 0 대신 0.1로 표시
    });
```

그래프 생성

- 그래프가 사용자의 요청에 따라 변동되어야 하므로
매번 삭제 후 재생성
- 구조분해 할당으로 각 그래프에 데이터 삽입

04

```
const chart = new Chart(ctx, {
  type: chartType,
```

- 그래프 생성 시 타입을 변수로 지정해 요청하는 타입에 맞게 생성

검색기능

- 클래스 토글방식 (클래스가 있으면 숨김 없으면 노출)
- 클릭한 태그 안에 한글,영어를
사용자가 입력한 내용과 비교

맵 생성 기능

- 서버에 데이터 요청과 로컬에 있는 파일을 병렬요청
- 받아온 데이터, 가져온 데이터를 기반으로
geojson기능 구현, 마커클러스터 생성

01 래프 타입 변경 함수

```
function changeGraphType(type) {
  chartType = type;
  let dateOnly = getDateOnly();

  const selectedCountry = document.querySelector('.country-list .active').dataset.country;

  fetchDataAndRender(selectedCountry, dateOnly);
}
```

그래프 생성

- 클릭한 요소에 active 클래스를 사전에 추가하고 active 클래스가 있는 태그의 추출한 나라 정보와 현재 사용자가 보고 있는 날짜를 함께 전송하여 차트를 생성.

```
# 기간별 데이터 조회
query = db.session.query(
  func.sum(WhoData.new_cases).label('new_cases'),
  func.sum(WhoData.new_recoveries).label('new_recoveries'),
  func.sum(WhoData.new_deaths).label('new_deaths')
).filter(
  WhoData.date_reported.between(start_date, end_date),
  WhoData.country == country
).first()
```

서버에서는 받은 나라와 날짜 기준으로
일간,주간,월간 데이터를 조회하여 반환

02

보는 화면에 맞게 지도크기 제어

```
function adjustMiddleContentHeight() {
  const mapBox = document.querySelector('#map-container');

  const dailyData = document.querySelector('.world-wide-daily');
  // const nav = document.querySelector('nav');
  const graphData = document.querySelector('.graph-box')

  const countryList = document.querySelector('.country-list ul');

  const updateBox = document.querySelector('.update-day');
  const searchBox = document.querySelector('.search-box');

  // 화면 전체 높이에서 헤더,일간현황,그래프의 높이를 뺀 값 계산
  const availableHeight = window.innerHeight - dailyData.offsetHeight - graphData.offsetHeight - 25;
  const availableHeightLeft = window.innerHeight - updateBox.offsetHeight - searchBox.offsetHeight - 25;
  mapBox.style.height = `${availableHeight}px`;
  countryList.style.height = `${availableHeightLeft}px`;
}

// 페이지 로드와 리사이즈 시 실행
window.addEventListener('load', adjustMiddleContentHeight);
window.addEventListener('resize', adjustMiddleContentHeight);
```

화면 제어 함수

- 화면 크기가 변동되어도 꽉찬 화면을 유지하기 위해 높이가 변경되어도 자연스럽게 상관없는 지도,국가리스트 태그의 높이를 제어

01

```
// 서버로 날짜 전송 (GET 방식)
fetch(`/worldwide/data?date=${selectedDate}`)
.then(response => {
  if (!response.ok) throw new Error('서버 요청 실패');
  return response.json(); // JSON 응답을 반환받음
})
.then(data => {
  // console.log("서버로부터 받은 데이터:", data);
  updateDOM(data); // 받은 데이터를 DOM에 반영

  // 선택트 박스 값
  dateSelect = document.querySelector('#date-select').value;
  // 일일 전세계 데이터 갱신
  fetchCovidData(dateSelect, selectedDate)
  // 차트 갱신
  changeGraphType('pie')
```

- 날짜 input에 입력된 날짜를 쿼리스트링으로 서버에 요청

- 받아온 데이터를 html에 갱신

(기존에 render_template로 데이터를 주고받는 라우트에서 값을 변경하여 반환하더라도 페이지가 다시 렌더링되지 않으므로, 직접 DOM을 제어함)

- 현재 선택트박스의 값을 받아 전세계 일간 데이터 갱신

-바꾼 날짜로 그래프 데이터 재요청 및 재생성

```
@worldwide_bp.route('/data', methods=['GET'])
def worldwide_data_json():
    selected_date = request.args.get('date', datetime.datetime.now().date() - datetime.timedelta(days=365))
    if isinstance(selected_date, str):
        selected_date = datetime.datetime.strptime(selected_date, '%Y-%m-%d').date()

    records, country_percentages, _ = get_covid_map_and_data(selected_date)

    return jsonify({
        'records': [
            {
                'country': record[0].country,
                'country_korean': record[1].country_korean,
                'new_cases': record[0].new_cases,
                'new_deaths': record[0].new_deaths,
                'cumulative_cases': record[0].cumulative_cases,
                'cumulative_deaths': record[0].cumulative_deaths,
                'lat': record[2].country_lat,
                'lng': record[2].country_long,
            }
            for record in records
        ],
        'country_percentages': country_percentages,
        'date_reported': records[0][0].date_reported.strftime('%Y-%m-%d') if records else None
    })
```

- 데이터를 받을때 정상이면 그대로 받고 잘못된 데이터면 2년6개월 전 날짜로 변경

- 문자열 데이터를 날짜형식으로 변경

- 데이터 조회 함수에 날짜를 보내 데이터 호출

- 화면에 보여줄 데이터들의 형식에 맞게 return

01

```

get_covid_map_and_data(selected_date=None):
    current_date = datetime.datetime.now().date()
    two_years_ago = current_date - datetime.timedelta(days=365 * 2 + 180)
    # selected_date가 제공되었으면 해당 날짜로, 아니면 기본값 두 년 반 전 날짜 사용
    if selected_date:
        # selected_date가 문자열이라면 날짜로 변환, 이미 datetime.date 객체라면 그대로 사용
        if isinstance(selected_date, str):
            selected_date = datetime.datetime.strptime(selected_date, '%Y-%m-%d').date()
        else:
            selected_date = two_years_ago

    records = db.session.query(WhoData, CountryTranslation, WorldLatLong).filter(
        WhoData.date_reported == selected_date,
        WhoData.country_code == CountryTranslation.country_code,
        WhoData.country_code == WorldLatLong.country_code
    ).distinct(WhoData.country).all()

    total_new_cases = sum(record[0].new_cases for record in records)

    country_percentages = []
    for record in records:
        who_data = record[0]
        country_translation = record[1]

        if total_new_cases > 0:
            percentage = (who_data.new_cases / total_new_cases) * 100
            country_percentages.append({
                'country': who_data.country,
                'country_korean': country_translation.country_korean,
                'percentage': round(percentage, 2)
            })

```

02

```

# 데이터 삽입 라우트
@worldwide_bp.route('/insert-data')
def insert_data():
    insert_data_to_db()
    return "데이터 삽입 작업이 완료되었습니다!"

@worldwide_bp.route('/insert-trans-data')
def insert_trans_data():
    insert_country_translations()
    return "번역 작업 완료"

@worldwide_bp.route('/insert-latlong-data')
def insert_latlong_data():
    insert_data_to_db_lat_long()
    return "위,경도 데이터 삽입 작업이 완료되었습니다!"

```

```

def insert_data_to_db():
    # CSV 파일 경로
    csv_path = os.path.join(os.getcwd(), 'apps/worldwide/static/data/WHO COVID-19 global daily data.csv')

    # CSV 파일 읽기
    data = pd.read_csv(csv_path)

    # 필수 컬럼 값 비어 있는 행 제거
    data = data.dropna(subset=['Date_reported', 'Country_code'])
    print(f"총 {len(data)}개의 데이터가 로드되었습니다.")

    bulk_insert_data = []
    bulk_update_data = []

    # 데이터 분리 (삽입 vs 업데이트용)
    existing_records = {
        (record.date_reported, record.country_code): record
        for record in WhoData.query.with_entities(WhoData.date_reported, WhoData.country_code).all()
    }

    for idx, row in data.iterrows():
        new_cases = int(row['New_cases']) if not pd.isna(row['New_cases']) else 0
        cumulative_cases = int(row['Cumulative_cases']) if not pd.isna(row['Cumulative_cases']) else 0
        new_deaths = int(row['New_deaths']) if not pd.isna(row['New_deaths']) else 0
        cumulative_deaths = int(row['Cumulative_deaths']) if not pd.isna(row['Cumulative_deaths']) else 0
        new_recoveries = int(row['New_Recoveries']) if not pd.isna(row['New_Recoveries']) else 0
        cumulative_recoveries = int(row['Cumulative_Recoveries']) if not pd.isna(row['Cumulative_Recoveries']) else 0

        country_code = row['Country_code'] if not pd.isna(row['Country_code']) else 'Unknown'
        country = row['Country'] if not pd.isna(row['Country']) else 'Unknown'
        who_region = row['WHO_region'] if not pd.isna(row['WHO_region']) else 'Unknown'

        key = (row['Date_reported'], country_code)

```

```

    if key in existing_records:
        bulk_update_data.append({
            "key": key,
            "new_cases": new_cases,
            "cumulative_cases": cumulative_cases,
            "new_deaths": new_deaths,
            "cumulative_deaths": cumulative_deaths,
            "new_recoveries": new_recoveries,
            "cumulative_recoveries": cumulative_recoveries,
        })
    else:
        bulk_insert_data.append(
            WhoData(
                date_reported=row['Date_reported'],
                country_code=country_code,
                country=country,
                who_region=who_region,
                new_cases=new_cases,
                cumulative_cases=cumulative_cases,
                new_deaths=new_deaths,
                cumulative_deaths=cumulative_deaths,
                new_recoveries=new_recoveries,
                cumulative_recoveries=cumulative_recoveries
            )
        )

```

- 해당하는 주소 입력시 데이터가 db에 삽입되는 함수 생성
- 처음 db 삽입 시 bulk_insert_data = []에 데이터 삽입
- 이후 db 삽입 시 bulk_update_data = []를 활용하여 기존 데이터는 update, 신규데이터는 insert방식으로 구동

- 날짜가 없으면 기본 2년 6개월 전으로 세팅

- 날짜에 따른 쿼리 조회

(3개의 table을 join하며 나라 중복을 제거하고 모두 반환)

- 모든 나라의 확진자 데이터를 더하고 자기 나라의 확진자를 나눠 비율을 계산하여 반환함

06 코드리뷰 - 국내

```
test = folium.utilities.JsCode("""
function(feature, layer) {
    layer.on('click', function(e) {
        let area = feature.properties.CTP_KOR_NM;
        if (area) {
            window.parent.location.href = '/' + encodeURIComponent(area);
        }
    });
});
""")
```

```
folium.GeoJson(
    geo_str,
    name='지역별 데이터',
    style_function=lambda feature: {
        'weight': 0.1,
        'color': 'transparent',
    },
    highlight_function=lambda feature: {
        'weight': 1,
        'fillOpacity': 0.4,
        'color': 'transparent',
    },
    popup=None,
    tooltip=folium.features.GeoJsonTooltip(
        fields=['CTP_KOR_NM', 'confirmed', 'deaths'], # 표시할 데이터
        aliases=['지역 이름:', '누적 확진자:', '누적 사망자:'], # 레이블
        localize=True,
        labels=True,
        sticky=True
    ),
    onEachFeature = test
).add_to(map)
```

```
map_html = map._repr_html_()
```

Folium.utilities.JsCode를 이용해 특정 지역을 눌렀을 때 해당 지역의 URL로 넘어갈 수 있게 함.

- 지도에 페이지 이동 기능이 심어진 상태로 지도가 생성됨.
- window.parent.location.href라고 해야 페이지 전체가 이동함.

Tooltip 기능 추가

- 마우스가 호버 됐을 때 tooltip이 나오게 함.
- json 파일에 넣어줬던 정보 활용

만들어진 지도 html 형식으로 저장.

06 코드리뷰 - 국내

```
path = os.path.join('apps', 'static', 'data', '국내_코로나_발생_현황.xlsx')
covid_data = pd.read_excel(path, sheet_name=None)
```

```
p_path = os.path.join('apps', 'static', 'data', '연령별_인구수.xlsx')
population_data = pd.read_excel(p_path)
```

```
sheet_data = { sheet : data for sheet, data in covid_data.items() }
```

```
@bp.route('/<graph_id>', methods=['GET'])
def get_graph(graph_id):
    if graph_id == 'area_incidence':
        area = sheet_data.get('시군구별(발생률, 사망률)')
        area_total = area.query("시군구=='합계'")
        data = {
            "labels": area_total['시도명'].tolist(),
            "values": area_total['발생률\n(인구10만명당, 명)'].tolist(),
            "label": '지역별 코로나 발병률(인구 10만명당, 명)',
            "type": "bar",
            "backgroundColor": "rgba(101,148,189,1)",
            "pointStyle": "circle",
            "borderColor": 'rgba(101,148,189,1)'
        }
    return jsonify(data)
```

필요한 엑셀 데이터들을 불러옴.

- 엑셀 파일 경로를 못 잡아서 os.path.join으로 경로 잡아줌.

엑셀 파일 하나가 시트가 여러 개라 딕셔너리 형태로 저장 -> 시트 이름으로 데이터 불러올 수 있게끔 처리

그래프로 그릴 모든 데이터를 같은 방식으로 처리

- 각각 그래프 아이디 지정.
- 필요한 데이터만 뽑아냄.
- Data라는 이름의 딕셔너리 형태로 그래프로 그릴 데이터, 그래프 형태, 스타일 등을 지정하여 만듦.
- Json 형태로 보냄.

06 코드리뷰 - 국내

```
const graphConfigs = [
  {id: "chart-area", url: "/api/graph/area_incidence"},
  {id: "chart-age", url: "/api/graph/age_incidence"},
];

const graphConfigsWithArea = [
  {id: "chart-time", url: `/api/graph/time_incidence/${area}`}
];
```

처음 페이지에 들어갔을 때 나타나는 그래프들의 데이터들을 지역페이지인지 아닌지 구분하여 딕셔너리 형태로 만들어줌.

```
const selectedConfigs = area ? graphConfigsWithArea : graphConfigs
```

URL에 지역이름이 들어가 있는지 없는 지로 구분하여 실행되는 딕셔너리 달라짐.

```
selectedConfigs.forEach(config => {
  fetch(config.url)
    .then(response => response.json())
    .then(data => {
      if (data.error) {
        console.error(`Error loading graph: ${config.url}`, data.error);
        document.getElementById(config.id).innerHTML = `<p>Error loading data</p>`;
      } else {
        createChart(config.id, data);
      }
    })
    .catch(error => {
      console.error(`Fetch error for ${config.url}:`, error);
      document.getElementById(config.id).innerHTML = `<p>Failed to load data</p>`;
    });
});
```

그래프를 만들기 위해 처리했던 데이터를 문제 없이 잘 받았다면 데이터들과 그래프의 id를 가지고 그래프를 만들어주는 함수로 이동.

```
function createChart(canvasId, data) {
  const ctx = document.getElementById(canvasId).getContext("2d");

  chartInstances[canvasId] = new Chart(ctx, {
    type: data.type,
    data: {
      labels: data.labels,
      datasets: [{
        label: data.label,
        data: data.values,
        backgroundColor: data.backgroundColor,
        borderColor: data.borderColor,
        pointStyle: data.pointStyle,
        borderWidth: 1.5
      }]
    },
    options: {
      responsive: false,
      scales: {
        y: {
          beginAtzero: true
        }
      },
      layout: {
        padding: {
          top: 90
        }
      }
    }
  });
}
```

그래프 생성해주는 차트

- chartInstances라는 비어있는 딕셔너리에 만들어진 그래프를 넣어줌.
- canvasId : html canvas 태그에다가 id 설정해준 걸로 어떤 그래프를 그릴지 정함.
- 데이터 처리하면서 json 형태로 보낸 자료 토대로 그래프 생성.

06 코드리뷰 - 국내

```
function changeChart(canvasId, button) {  
  const graphId = button.getAttribute("data-graph");  
  const url = area ? `/api/graph/${graphId}/${area}` : `/api/graph/${graphId}`;  
  if (graphId) {  
    fetch(url)  
      .then(response => response.json())  
      .then(data => {  
        if (chartInstances[canvasId]) {  
          chartInstances[canvasId].destroy();  
        }  
        createChart(canvasId, data);  
      })  
      .catch(error => console.error(`Error loading graph: ${graphId}`, error));  
  }  
}
```

```
if(!area) {  
  document.querySelector("#graph1 div").addEventListener("click", (event) => {  
    const canvasId = "chart-area";  
    const button = event.target;  
  
    changeChart(canvasId, button);  
  });  
  
  document.querySelector("#graph3 div").addEventListener("click", (event) => {  
    const canvasId = "chart-age";  
    const button = event.target;  
  
    changeChart(canvasId, button);  
  });  
}  
  
if(area) {  
  document.querySelector("#graph4 div").addEventListener("click", (event) => {  
    const canvasId = "chart-time";  
    const button = event.target;  
  
    changeChart(canvasId, button);  
  });  
}
```

버튼을 누르면 그래프 변경

- 버튼에 data-graph라는 속성 추가하여 graphId를 값으로 해줌.
- 버튼 누르면 기존에 chartInstances에 저장 됐었던 canvasId가 같은 그래프를 없애줌.
- 그래프 생성하는 함수로 넘어감.
- 거기서 만들어진 그래프가 다시 화면에 나타남.

만들어둔 각종 그래프 관련 함수들을 재사용하기 위한 코드

- 그래프를 만들기 위해서 각각의 canvasId랑 button을 다르게 지정해야 하기 때문

06 코드리뷰 - 국내



```
if area in ["충청북도", "충청남도", "전라북도", "전라남도", "경상북도", "경상남도"]:  
    short_area = area[0] + area[2]  
else:  
    short_area = area[:2]
```

```
url = f"http://apis.data.go.kr/1352000/ODMS_COVID_06/callCovid06Api?serviceKey=  
{api_key}&sid={short_area}"  
response = requests.get(url)  
data = xmltodict.parse(response.text)
```

```
try:  
    items = data['response']['body']['items']['item']  
    if not isinstance(items, list):  
        items = [items]  
  
    hospitals = [  
        {'name': item.get('hospitalNm'), 'tel': item.get('hospitalTel'), 'url':  
f"https://map.naver.com/p/search/{item.get('hospitalNm')}"},  
        for item in items  
    ]  
  
except KeyError:  
    hospitals = []
```

보건소 api 내에 지역 이름 두 글자
모든 지역 페이지의 지역 이름은 두 글자가 아님.
-> 이러한 문제를 해결

해당 api 내에서 필요한 정보만 추출
-> 필요한 정보를 딕셔너리 형태로 저장.
-> html에서 표형태로 출력

```
document.addEventListener('DOMContentLoaded', () => {  
  const sidebarLinks = document.querySelectorAll('.domestic_sidebar a');  
  const sections = document.querySelectorAll('article');  
  
  const observerOptions = {  
    root: null,  
    rootMargin: '0px',  
    threshold: 0.9  
  };  
});
```

```
const observer = new IntersectionObserver((entries) => {  
  entries.forEach(entry => {  
    const id = entry.target.id;  
    const sidebarLink = document.querySelector(`.domestic_sidebar a[href="#${id}"]`);  
  
    if (entry.isIntersecting) {  
      sidebarLinks.forEach(link => link.classList.remove('active'));  
      if (sidebarLink) {  
        sidebarLink.classList.add('active');  
      }  
    }  
  });  
}, observerOptions);  
  
sections.forEach(section => observer.observe(section));  
});
```

내가 지정한 구역이 90% 이상 보일 때 해당 영역을 보고 있다고 표시 됨.

Intersection observer 라이브러리 사용.

이미 active가 붙어있으면 제거해줌.

후에 내가 정한 구역의 90% 이상이 보일 때 해당 구역에 'active' 라는 클래스 이름이 붙음.

그러면서 사이드바에 내가 보고 있는 위치가 어딘지 표시가 됨.

<COVID-19 완치율>

```

@bp.route('/api/covid-summary', methods=['GET'])
def get_covid_summary():
    # CSV 파일 경로 설정
    csv_path = os.path.join(os.getcwd(), 'apps/static/data/코로나격리해제.csv')

    # CSV 파일 읽기
    data = pd.read_csv(csv_path)

    # "검역" 값을 제외
    data = data[data["region"] != "검역"]

    # 제공된 데이터프레임 형태로 변환
    # 그룹화하여 지역별로 합계 계산
    df = data.groupby("region").agg(
        |   확진자=("confirmed", "sum"),
        |   사망자=("death", "sum"),
        |   격리해제=("released", "sum"),
    ).reset_index()

    # 열 이름 변환
    df.columns = ["지역", "확진자", "사망자", "격리해제"]
    df = df.copy()
    df["완치율(%)"] = (df["격리해제"] / df["확진자"] * 100).round(2)
    df["완치율(%)"] = df["완치율(%)"].fillna("N/A") # 결측값 처리

    table_data = df.to_dict(orient="records") # 데이터를 딕셔너리 리스트로 변환
    # 데이터를 딕셔너리 리스트로 변환
    table_data = df.to_dict(orient="records")
    print(table_data)

    # 데이터를 JSON 형태로 반환
    return jsonify(table_data)

```

Views 부분 코드.

- 1.covid-summary api를 불러옴.
- 2.판다스로 엑셀파일을 가져옴.
- 3.지역에 검역이라는 부분이 있었는데, 검역부분만 그래프 차이가 많이나서 지역그래프에 어울리지 않아서, 검역부분만 제외시킴.
- 4.지역, 확진자, 사망자, 격리해제 이부분으로 계산이 되기 때문에 그룹화해서 묶어놓음.
- 5.완치율은 %지로 나와야하기 때문에 격리해제 / 확진자 *100 으로 계산
- 6.table_data를 딕셔너리로 변환.
- 7.JSON형태로 변환
8. 하영님이 전체적으로 만든 코드에 완치율부분은 추가로 넣어서 작업했고, 완치율에 대한 정보가 딱히 없어서 선생님이 보내주신 엑셀파일 자료로 진행.

격리해제를 완치율로 보고 작업함.

06 코드리뷰 - 국내

```
fetch('/domestic/api/covid-summary')
  .then(response => response.json())
  .then(data => {
    // 데이터 처리
    const labels = data.map(item => item["지역"]);
    const recoveryRates = data.map(item => item["완치율(%)"]);

    // Chart.js로 그래프 생성
    const ctx = document.getElementById('recoveryRateChart').getContext('2d');
    new Chart(ctx, {
      type: 'line',
      data: {
        labels: labels, // x축 레이블
        datasets: [{
          label: '완치율 (%)',
          data: recoveryRates, // y축 데이터
          backgroundColor: 'rgba(75, 192, 192, 0.2)',
          borderColor: 'rgba(75, 192, 192, 1)',
          borderWidth: 5
        }]
      },
      options: {
        responsive: true,
        scales: {
          y: {
            beginAtZero: true,
            max: 10,
            title: {
              display: true,
              text: '완치율 (%)'
            }
          },
          x: {
            title: {
              display: true,
              text: '지역'
            }
          }
        }
      }
    });
  })
  .catch(error => console.error('Error fetching data:', error));
```

```
<article id="graph2">
  <h3>지역별 코로나 완치율</h3>
  <div class="recoveryRateChart">
    <canvas id="recoveryRateChart" width="800" height="500"></canvas>
  </div>
</article>
```

그래프 코드부분.(Chart.js 사용)

- 1.covid-summary를 가져와서 지역과 완치율로 데이터 처리함
2. Chart.js로 그래프를 생성, x축과 y축으로 나눠서 지역과 완치율에대한 그래프를 만듦
- 3.그래프로 표현함에 있어서 지역별 완치율은 %가 높지 않았기 때문에 치수를 10으로 줄여서 진행했고,

처음엔 원형그래프로 진행하려 했으나 지역별로 나온 %가 큰차이가 없어서 선그래프로 변경하고 보이는 그래프 선 두께도 조절해서 작업했습니다.

Index부분

recoveryRateChart를 불러와서 기존에 만들어져있던 Index에 그래프2로 추가함.

06 코드리뷰 - 국외

```
function searchCountryData(country) {  
  fetch(`/crw/crawl?country=${encodeURIComponent(country)}`)  
    .then(response => response.json())  
    .then(data => {  
      if (data.status === "success") {  
        displaySearchResults(data);  
      } else {  
        searchResultsDiv.innerHTML = "<p>데이터를 가져올 수 없습니다.</p>";  
      }  
    });  
}
```

- 
- DOMContentLoaded 이벤트 리스너를 통해 초기화
 - 국가 선택 이벤트 핸들러에서 AJAX 호출
 - 서버 응답 데이터를 기반으로 UI 업데이트

```
@crw_bp.route('/crawl', methods=['GET'])  
def crawl_data():  
    country = request.args.get('country')  
    if not country:  
        return jsonify({"status": "error", "message": "Country parameter is missing."})  
  
    try:  
        news = search_naver(f"{country} 코로나 뉴스")  
        return jsonify({"status": "success", "result": news}), 200  
    except Exception as e:  
        return jsonify({"status": "error", "message": str(e)}), 500
```

- 
- /crw/crawl 라우트에서 국가별 크롤링 데이터 반환
 - Search_naver 및 search_google 함수로 뉴스 데이터 수집

```
import requests
from bs4 import BeautifulSoup

def search_naver(query):
    url = f"https://search.naver.com/search.naver?query={query}"
    headers = {'User-Agent': 'Mozilla/5.0'}
    response = requests.get(url, headers=headers)

    if response.status_code != 200:
        print(f"Error: HTTP {response.status_code}")
        return []

    soup = BeautifulSoup(response.text, 'html.parser')
    news = []

    for item in soup.select('.total_wrap'):
        title_tag = item.select_one('.total_tit')
        a_tag = title_tag.select_one('.link_tit')

        if title_tag:
            title = title_tag.get_text(strip=True)
            link = a_tag.get('href', '#')
            metadata_tag = item.select_one('.name')
            metadata = metadata_tag.get_text(strip=True) if metadata_tag else "정보 없음"

            news.append({
                'title': title,
                'url': link,
                'metadata': metadata
            })

    return news
```



- 네이버 뉴스 크롤러 코드:
- 네이버 검색 URL 생성 및 BeautifulSoup 파싱
- 뉴스 제목, 링크, 메타데이터 추출 후 리스트로 반환

06 코드리뷰 - 국외

javascript

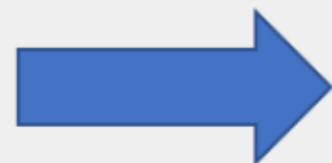
```
selectedCountryDiv.textContent = `선택된 국가: ${initialCountry}`;  
searchCountryData(initialCountry);
```



..코멘트: 초기값 설정 및 API 호출.

javascript

```
countryList.addEventListener("click", (event) => {  
  const li = event.target.closest('li');  
  if (li) {  
    const country = li.getAttribute('data-country');  
    if (country) {  
      selectedCountryDiv.textContent = `선택된 국가: ${country}`;  
      searchCountryData(country);  
    }  
  }  
});
```



코멘트: 사용자 이벤트 처리 및 데이터 동기화.

BackEnd



Flask
Python [3.11.9]

FrontEnd



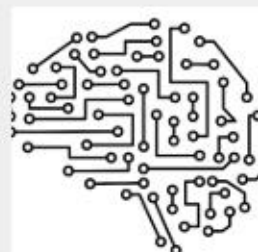
HTML
CSS
JavaScript

DataBase



Mysql

AI / ML



Keras
TensorFlow
scikit-learn
joblib

지도 시각화



Leaflet
GeoJSON
Folium

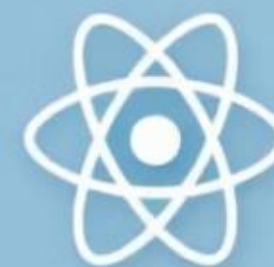
차트 시각화



Chart.js



+



+

PostgreSQL



1. 지도 라이브러리 폴리움 호출 시 전역적으로 css를 건드려 기존 레이아웃 세팅이 무너짐

원인: 기존에는 html을 변환해서 넣었고 부트스트랩이 기본 html을 직접적으로 건드려 기존css들에 영향을 끼침

해결: html통으로 넣어서 간섭이 없게끔 변경

2. 위,경도 DB등록 함수 테스트 중 NaN오류발생, 데이터도 직접 펼쳐보고, isna를 써서 NaN값을 0으로 기본세팅 했는데도 오류가 발생

원인: NA라는 국가코드때문에 NA를 nan으로 인식함

해결: NA국가 코드 대신 겹치지 않는 국가 코드 임의 기입

3. 셀렉트 박스 변경 시 데이터 처리속도가 너무 늦음

원인: 대용량(40만개) 데이터를 쿼리조회해서 뿌려주는 기능이라 무겁고 잦은 반복문 사용과, 꺼내 온 후 계산처리 로직

해결: 반복문과 계산을 최대한 줄이고 쿼리문에서 가져올때 미리 계산 처리 후 결과값만 가져옴 → 그래도 1초 정도의 딜레이가 있어 받아오는 시간동안 로딩중 화면이 보이게끔 대처

2차해결: index를 같이 넣어줘서 조회시간 단축

4. 모델 생성 및 모델을 기존 코드와 결합하는 과정

원인: 모델 호출 시 날짜만 넣어주면 될줄 알았는데 정규화가 필요함, 신규 정규화 방식 적용 시 예측 데이터 부정확 | 반환받는 형태가 서로다름

해결: 모델 생성할때 정규화하는 X,y도 파일로 같이 생성 후 호출하는곳에서 불러 사용함 | Decimal형태로 받아서 소수점도 교정처리함

5. 완치율 자료 부족

문제: 확진자, 사망자, 격리해제 자료들은 많았으나 완치율은 따로 없었고 구하려면 직접 계산을 해야하는 번거로움이 있었다.

해결: 선생님께 도움을 요청했고 자료를 주셨으나 완치율에대한 자료는 아니였고 격리해제를 완치율로 보고 하라고 하셔서 격리해제를 기준으로 작업을 했습니다.

6. 그래프 만드는 코드를 재사용하고 싶음.

원인: 그래프가 들어가는 자리가 모두 다르고 그래프가 만들어지는데 사용되는 데이터와 만들어지는 그래프가 모두 다름. 해당 그래프가 나오는 페이지도 달라야 함.

해결: 조건문을 달아 해당 그래프가 제공될 곳이 메인페이지인지 지역 페이지인지 구분. html 그래프가 들어갈 구역과 canvas 태그와 button에 id를 지정.

해당 id에 따라 만들어지는 그래프가 달라지게 구현.

7. 버튼을 눌렀을 때 그래프 교체가 안 됨.

원인 : chartInstances라는 딕셔너리에 만들어진 그래프를 초기화 시킨 뒤 변경될 그래프가 들어가야하는데 안됨.

해결: chartInstances안의 값들이 인식이 안 되는 문제를 해결하기 위해 아예 addEventListener 바깥쪽으로 뺌.

8. 지도에서 특정 구역을 눌렀을 때 전체적인 페이지 이동이 안 되고 네모난 검정색 선이 생김.

원인 : 지도가 iframe 형식임.

해결: 만들어질때 파이썬에서 css코드와 js코드를 사용해서 검정색 선을 없애고 페이지를 이동할 수 있는 기능이 심어진 상태로 지도를 만들.

또한 지도가 있는 자리만 페이지가 이동되는 문제를 해결하기 위해 window.parent.location.href를 사용.

9. 해당 지역의 병원이 마커로 찍혀있는 지도

원인 : 폴리움 이용 -> 페이지 로딩 속도 3-5분, 모든 보건소가 다 안 찍힘. 카카오맵 이용 -> 카카오 정책이 바뀌어서 사용하려면 3-5일 정도 심사일이 걸림. 시간 촉박.

해결: 표로 대체하고 병원 이름에 href를 걸어 해당 병원이 검색된 네이버지도 결과화면으로 이동하게끔 구현

10. 데이터 불완전성 문제

문제: 일부 국가의 데이터가 누락되거나 크롤링된 결과가 불완전.

원인: 크롤링 대상 웹사이트의 구조 변경. 데이터 접근 권한 제한(예: robots.txt).

해결 방안

크롤링 코드 업데이트: 크롤링 대상 웹사이트의 구조 변경에 따라 XPath/Selector 수정.

데이터 누락 방지: 크롤링 중 누락된 데이터를 로그로 기록하여 추적 및 재처리.

robots.txt 체크: 크롤링 대상 사이트가 접근 가능한지 사전 확인.

결과 요약:

크롤링 오류 발생 빈도 감소 (데이터 불완전성 개선).

사용자 경험 개선 (로딩 상태와 오류 메시지 명확화).

UI 성능 개선 (페이징 및 로딩 최적화).

전체 응답 속도 향상 (Fetch 재시도 및 타임아웃 설정).

11. 비동기 통신 문제

문제: Fetch 요청이 실패하거나 서버에서 응답 시간이 길어지는 경우, 사용자 경험 악화.

원인: 서버 부하. 네트워크 연결 문제.

해결 방안: Fetch API의 타임아웃 설정: 요청 시간이 길어질 경우, 타임아웃으로 처리. 백오프(backoff) / 재시도 로직 추가: 네트워크 문제 발생 시, 최대 3회까지 재시도.

12. UI 비효율성 문제

문제: 검색 결과가 많아질 경우 사용자 인터페이스(UI)가 과부하.

원인: 모든 결과를 한 번에 렌더링. 페이징 또는 무한 스크롤 기능 미흡.

해결 방안: 페이징 기능 도입: 한 번에 표시하는 결과 수 제한(예: 페이지당 10개).

로딩 표시 개선: 사용자에게 명확한 로딩 상태 제공.

최적화: 가상 스크롤링(virtual scrolling) 활용.

최하영

페이지에서 기능들을 구현하면서 어떤 코드와 함수를 만들면 좋을지 등이 금방 떠오르지 않았다.

또한 배운 것임에도 불구하고 조금만 응용하면 될 것 같았던 것들이 갈피가 잡히지 않아 찾아보고 이해하고 구현하느라 간단한 기능들도 시간이 오래 걸렸던 것 같다.

하지만 프로젝트를 수행하며 문제가 생기면 어떻게 확인해야하는지 그에 대한 방법을 조금은 깨달았다.

그리고 기능 하나를 구현하는데 시간을 얼마나 들건지 정해놓고 하니까 해야할 일이 정리되고 시간이 갈수록 과제 진행이 엄청 지체된다는 생각은 안 들었다. 이런 것들을 참고하여 추후에는 좀 더 많은 것들을 할 수 있도록 해야겠다.

권민수

배웠던 내용이지만
그래도 미숙한부분이 많이 있었다...
어떻게 시작을 해야할지, 어떻게 정리를 해야할지,
어떻게 구현할지 막막했지만...
그래도 주위에 도움으로 인해
진행할수 있었습니다.
부족한부분이 너무 많다고 느꼈고,
더 많이 공부하고 알아야겠다고 느꼈다.

이영찬

지난 프로젝트 경험을 바탕으로, 팀의 실력에 맞는 적절한 난이도로 진행한 덕분에 일정 관리에 여유가 생겼습니다. 특히, 기간별로 어떤 기능을 완성할지 계획을 세운 것이 작업 속도 조절에 큰 도움이 되었습니다.

아쉬운점으로 담당했던 기능 중 날짜 선택 기능 추가를 고려하지 않고 동작만 되는 코드를 작성하다 보니, 기능을 포기하거나 처음부터 다시 짜야 되나 싶었던 적이 있습니다. 다행히 아이디어가 떠올라 간신히 구현했지만, 코드의 가독성과 반복적인 호출 방식은 만족스럽지 못했습니다.

이 경험을 통해, 다음 프로젝트에서는 기능 구현 전에 더 깊이 고민하고, 확장성을 포함한 설계를 철저히 준비한 후 작업을 시작해야겠다는 교훈을 얻었습니다.

허경

처음해보는 기능을 구현하며
쉬운줄 알았지만
모르는 부분이 너무 많았습니다
이번프로젝트를 하면서
많이 공부했습니다

2025—
Happy new year!

2024.12.30 MBC 아카데미 2조

이상 발표를 마치겠습니다

감사합니다